Diccionarios

Estadística con Python

Mario A. Morales Rivera
mamorales@correo.unicordoba.edu.co
Universidad de Córdoba



Tabla de contenidos I

1. Introducción
2. Creación de diccionarios
3. Edición de diccionarios
4. Claves y valores a listas
5. Iteración sobre diccionarios
6. Métodos aplicables a diccionarios
7. Aplicación



Introducción



Diccionarios

Nota

Un diccionario es un objeto muy flexible de Python para almacenar diferentes tipos de datos y objetos, tales como números, caracteres, listas y arreglos.

- Como estudiamos en clase anterior, una lista es una colección de objetos indexada desde cero (0) hasta el número de objetos menos uno $(n-1)$.
- En lugar de buscar un elemento a través de un índice de números enteros puede ser más fácil, práctico (o útil) usar una cadena de texto con un nombre, que sea fácil de recordar.



Diccionarios (cont)

- Los diccionarios nos permiten identificar cada elemento por una clave (`key`) que por lo general es un *string* o cadena de caracteres.
- Rigurosamente hablando, un diccionario en Python es una lista donde los índices pueden ser cadenas de texto.
- Suponga que necesitamos guardar las edades de tres personas: Brenda, Luis y Diego. Para eso podemos usar una lista

```
edad=[22,15,19]
```



Diccionarios (cont)

- Pero de esta forma debemos recordar la secuencia de nombres, por ejemplo que el índice cero corresponde a Brenda, el índice 1 a Luis y el 2 a Diego.
- Esto es, la edad de Luis se obtiene como `edad[1]`. Un diccionario con los nombres de las personas como índice es más conveniente porque esto permite escribir `edad['Luis']` para recuperar la edad de Luis, sin tener que recordar el índice numérico.



Creación de diccionarios



Creación de diccionarios

En esta sección se estudian las formas de crear diccionarios, primero mediante `{clave:valor,...}` luego mediante la función `dict`

A continuación se crea un diccionario que contiene los nombres de las personas como claves y sus edades como valores

```
edad={'Luis':15,'Brenda':22,'Diego':19}
```

Creación de diccionarios: función `dict()`

De manera equivalente, podemos usar la función `dict()` para crear el mismo diccionario. En este caso entregamos los datos de la forma `clave=valor` como se muestra a continuación

```
edad=dict(Luis=15,Brenda=22,Diego=19)
```

Creación de diccionarios: `dict(zip())`

- Suponga que tenemos los nombres de las personas en una lista o tupla y sus edades en otra lista o tupla, podemos crear el diccionario mediante

Edición de diccionarios



Adición de elementos a un diccionario

En cualquier momento, después de su creación, se puede adicionar una pareja `clave:valor` a un diccionario, por ejemplo, a partir del diccionario `edad` agregamos a `Jaime` , cuya edad es 21 años

```
edad["Jaime"]=21
print(edad)
```

```
{'Luis': 15, 'Brenda': 22, 'Diego': 19, 'Jaime': 21}
```



Diccionarios: tipos de valores

En el ejemplo anterior todos los valores del diccionario son números enteros, pero es posible crear diccionarios con diferentes tipos de valores, como en el siguiente ejemplo

```
d1= {
     "clave1":234,
     "clave2":True,
     "clave3":"Valor 1",
     "clave4":[1,2,3,4]}
```



Diccionarios: tipos de valores (II)

como con los objetos estudiados en las secciones anteriores, podemos usar la la función `type()` para confirmar si un objeto de Python es un diccionario y obtener el tipo de datos que tienen sus elementos

```
type(d1)                          ①
type(d1) is dict                  ②
type(d1['clave2']) is int         ③
```

① Regresa `dict` si `d1` es un diccionario.
② Regresa `True` si `d1` es un diccionario, en caso contrario regresa `False`.
③ Regresa `True` si el valor correspondiente a `clave2` es un entero, en caso contrario regresa `False`.

```
<class 'dict'>
True
False
```



Acceso a claves y valores de un diccionario

Para verificar si una clave está en el diccionario `d` usamos `clave in d`, lo cual regresará un valor lógico, `True` o `False` dependiendo de si la clave está o no en el diccionario. Para recuperar el valor asociado a la clave de un diccionario usamos `d['clave']`, veamos

```
'Mary' in edad                    ①
edad["Jaime"]                     ②
```

① Regresa `True` si la clave `Mary` está en el diccionario `edad`, en caso contrario, regresa `False`.
② Regresa el valor asociado a la clave `Jaime`

```
False
21
```



Diccionarios: KeyError

Si usas la forma `d['nombre']` y `nombre` no es una clave del diccionario, ocurre un `KeyError`. Para evitarlo, podemos usar la sentencia `if` para tomar decisiones, por ejemplo: en caso que se tenga el dato de `Mary` en el diccionario se imprime su edad, de lo contrario, se imprime un mensaje informando que ese nombre no se encuentra

```
nombre='Mary'                                                    ①
if nombre in edad:                                               ②
  print("La edad de %s es %g" % (nombre,edad[nombre]) )          ③
else:                                                            ④
  print("No hay datos para %s" % nombre )
```

① Se asigna `Mary` a nombre.
② Si `Mary` es una clave del diccionario edad.
③ Se imprime la clave y el valor.
④ En caso contrario, se imprime que no hay valor para esa clave.

```
No hay datos para Mary
```



Diccionarios control KeyError con try:except

Tip

Como se explicó en la clase de listas, otra forma de tratar con este error y evitar que el código reviente es mediante try:except

```
clave='Luis'
try:
    e= edad[clave]
    print(f"La edad de {clave} es {e} años")
except KeyError:
    print(f"{clave} no es una clave de edad")
```

```
La edad de Luis es 15 años
```



Claves y valores a listas



Extraer claves y valores en listas

Tanto las claves como los valores en un diccionario se pueden extraer y convertir a una lista, veamos

```
claves=edad.keys()                    ①
print(type(claves))                   ②
list(claves)                          ③
valores=list(edad.values())           ④
print(valores)                        ⑤
```

① Se recuperan las claves con el método `keys`.
② Note que `claves` es de tipo `dict_keys`.
③ Convertimos `claves` a una lista.
④ Recuperamos los valores con el método `values` que es de tipo `dict_values` y lo convertimos a lista.
⑤ Imprimimos los valores.

```
<class 'dict_keys'>
['Luis', 'Brenda', 'Diego', 'Jaime']
```



Extraer claves y valores en listas

- Note que `claves`, el objeto regresado por el método `.keys()`, no es una lista, pero podemos recuperar las claves como lista con la función `list()`.
- Análogamente con los valores del diccionario, usando el método `.values()` se obtiene un objeto de tipo `dict_values` que se puede convertir a lista, con la función `list()`.



Iteración sobre diccionarios



Iterar sobre un diccionario

De manera análoga a tuplas, listas y arreglos es posible iterar sobre las claves de un diccionario usando bucles, por ejemplo

```
for nombre in edad:
  print(f"La edad de {nombre} es {edad[nombre]} años")
```

```
La edad de Luis es 15 años
La edad de Brenda es 22 años
La edad de Diego es 19 años
La edad de Jaime es 21 años
```



Iterar sobre un diccionario (II)

Suponga que deseamos la salida anterior pero en orden alfabético, por los nombres, eso podemos lograrlo con la función `sorted()`, la cual regresa una lista con las claves ordenadas

```
sorted(edad)                                                    ①
for nombre in sorted(edad):                                     ②
  print(f"La Edad de {nombre} es {edad[nombre]} años")
```

① Lista con los nombres (claves del diccionario), en orden alfabético.
② Se recorren las claves, ordenadas alfabéticamente.

```
['Brenda', 'Diego', 'Jaime', 'Luis']
La Edad de Brenda es 22 años
La Edad de Diego es 19 años
La Edad de Jaime es 21 años
La Edad de Luis es 15 años
```



Iterar sobre un diccionario

Tenga en cuenta que la función `sorted()` no ordena ni modifica el diccionario, solo regresa una listas que contiene las claves ordenadas. En el siguiente bloque de código creamos un nuevo diccionario ordenado a partir de diccionario `edad`

```
edad_o={}                                          ①
for clave in sorted(edad):
    edad_o[clave]=edad[clave]                      ②
print(edad_o)                                      ③
```

① Creamos un diccionario vacío.
② Llenamos el diccionario vacío con las claves ordenadas previamente.
③ Imprimimos.

```
{'Brenda': 22, 'Diego': 19, 'Jaime': 21, 'Luis': 15}
```



Iterar sobre claves y valores: `.items()`

El método `.items()` devuelve una vista de los pares clave-valor del diccionario como tuplas. Esto es extremadamente útil para desempaquetar ambos elementos directamente en el bucle `for`:

```
for nombre, edad_persona in edad.items():
    print(f"La edad de {nombre} es {edad_persona} años")
```

```
La edad de Luis es 15 años
La edad de Brenda es 22 años
La edad de Diego es 19 años
La edad de Jaime es 21 años
```



Ordenar elementos de un diccionario

Alternativamente, podemos ordenar los elementos del diccionario por las claves usando el método `.OrderedDict()` del módulo `collections`.

```
import collections
EdadOr=collections.OrderedDict(sorted(edad.items()))
print(dict(EdadOr))
```

```
{'Brenda': 22, 'Diego': 19, 'Jaime': 21, 'Luis': 15}
```



Ordenar elementos de un diccionario

Observe que el ordenamiento se hace de manera ascendente, para hacerlo en orden descendente use `reverse=True` en la función `sorted()`



Hacer copia de un diccionario

Se puede hacer una copia de un diccionario mediante el método `.copy()` y como en el caso de las listas y arreglos, se puede borrar una entrada de un diccionario con `del`

```
edadC=edad.copy()                    ①
del edadC["Luis"]                    ②
edadC
```

① Copia del diccionario edad.
② Se borra de `edadC` la clave `Luis` junto con su valor.

```
{'Brenda': 22, 'Diego': 19, 'Jaime': 21}
```



Métodos aplicables a diccionarios



Método .pop()

La mayoría de los métodos que estudiamos para listas son aplicables a diccionarios.

El *método `pop()`* remueve específicamente una clave de diccionario y devuelve valor correspondiente. En caso que la clave dada como argumento no se encuentre en el diccionario, `pop()` lanza una excepción `KeyError`.

```
edadC=edad.copy()                    ①
edadC.pop("Luis")                    ②
edadC
```

① Copia del diccionario edad.
② Regresa la edad de Luis y lo elimina del diccionario.

```
15
{'Brenda': 22, 'Diego': 19, 'Jaime': 21}
```



Método .popitem()

Observe que en edadC ya no está la clave Luis.

El *método .popitem()* funciona como pop(), pero con la última clave, es decir, elimina el último ítem del diccionario y regresa una tupla con la clave y el valor correspondientes.

```
edadC=edad.copy() # copia de edad
edadC.popitem()
```

```
('Jaime', 21)
```



Método update()

El *método* `update` permite actualizar la información contenida en un diccionario. Recibe como parámetro otro diccionario: si los dos diccionarios tienen claves en común, actualiza los valores de las claves comunes; si no hay claves comunes, los pares `clave:valor` no comunes serán agregados al diccionario.

```
edad2={"Brenda":25,"Mary":18}                    ①
edad.update(edad2)                               ②
edad
```

① Nuevo diccionario, note que Brenda ya está en el diccionario edad
② Se actualiza el diccionario edad, observe que a Brenda, que es una clave común a los dos diccionarios, se le actualiza la edad.

```
{'Luis': 15, 'Brenda': 25, 'Diego': 19, 'Jaime': 21, 'Mary': 18}
```



Método .get()

El *método* `get()` recibe una clave y devuelve el valor que le corresponde en el diccionario.

El *método* `.setdefault()` tiene dos usos: el primero es como el método `get()` y, el segundo, para agregar un nuevo elemento al diccionario

```
edad.get("Brenda")                                    ①
edad.setdefault("Brenda")                             ②
edad.setdefault("Lila",16)                            ③
for clave in edad:
    print(f"'{clave}':{edad[clave]}")                  ④
```

① `get()`: recupera el valor de la clave Brenda.
② `setdefault`, primer uso: exactamente lo mismo que `get()`.
③ `setdefault`, segundo uso: agrega al diccionario `edad` una nueva pareja `clave:valor`, `Lila:16`.
④ Recorremos el diccionario para imprimir de forma vertical.



Tuplas a diccionarios

Considere la siguiente tupla, conformada por 6 tuplas anidadas:

```
tup = (('A', 5), ('A', 7),
       ('B', 2),('B', 9),('B', 1),
       ('C',4) )
```

Note que `'A'` está dos veces; `'B'`, 3 veces y `'C'` una vez en la tupla. El ejercicio consiste en trasladar esta información a un diccionario donde `'A'`, `'B'` y `'C'` conformen las claves y los valores que coresponden a cada clave se organicen en una lista.



Tuplas a diccionarios (cont)

```
dicn = {}                                    ①
for clave, valor in tup:                     ②
    if clave in dicn:                        ③
        dicn[clave].append(valor)            ④
    else:                                    ⑤
        dicn[clave] = [valor]                ⑥
dicn
```

① Diccionario vacío para guardar los datos.
② Se recorre la tupla.
③ En caso que la clave ya esté en dicn.
④ Agrega el valor a la lista
⑤ En caso que la clave aún no esté
⑥ Anexa a `dicn` la clave con el respectivo valor

```
{'A': [5, 7], 'B': [2, 9, 1], 'C': [4]}
```



Observe que la tupla inicial tiene longitud 6, mientras que la longitud del diccionario, que contiene la misma información es 3. Esta forma de guardar los datos es mucho más eficiente en cuanto al uso de memoria y espacio en disco duro. Otra forma de realizar la misma tarea es mediante la combinación de los métodos `.setdefault()` y `.append()`

```
dicn = {}                                                    ①
for clave, valor in tup:                                     ②
    dicn.setdefault(clave, []).append(valor)                 ③
dicn
```

① Diccionario vacío para guardar los datos.
② Se recorre la tupla.
③ Se agrega la información al diccionario: `setdefault(clave, [])` verifica si la clave no está la agrega con una lista vacía como valor, luego `append(valor)` agrega el valor a la lsita.

```
{'A': [5, 7], 'B': [2, 9, 1], 'C': [4]}
```



Comprensión de Diccionarios

Al igual que con las listas, podemos generar nuevos diccionarios de forma compacta utilizando una sola línea de código.

Supongamos que necesitamos calcular el cuadrado de las edades:

```
edades_cuadrado = {nombre: valor**2 for nombre, valor in edad.items()}
print(edades_cuadrado)
```

```
{'Luis': 225, 'Brenda': 625, 'Diego': 361, 'Jaime': 441, 'Mary': 324, 'Lil
```



Aplicación



Polinomios como diccionarios

Las claves en los diccionarios no tienen que ser necesariamente cadenas de caracteres *(strings)*, en realidad cualquier objeto **inmutable** (como las tuplas) de Python puede usarse como clave. En este ejemplo se muestra como los diccionarios con enteros como claves se pueden usar para representar polinomios. Los datos asociados con el polinomio

$$p(x) = -1 + x^2 + 3x^7 \tag{1}$$

se pueden expresar como un conjunto de pares de la forma `clave:valor`, donde la clave corresponde al exponente y el valor al coeficiente



Las entradas del diccionario, de acuerdo con esto, son {exponente:coeficiente}.

En el caso del polinomio dado en (1), el coeficiente -1 corresponde al exponente cero para la variable x, ya que $-1 = -1x^0$, el coeficiente 1 pertenece al exponente 2, el coeficiente 3 corresponde al exponente 7. A continuación se usa el diccionario p para guardar la información de (1)

```
p={0:-1, 2:1, 7:3}
for m in p:
        print(f"Coeficiente: {p[m]}, Exponente: {m}")
```

```
Coeficiente: -1, Exponente: 0
Coeficiente: 1, Exponente: 2
Coeficiente: 3, Exponente: 7
```



Podemos definir una función que recibe un polinomio (expresado como un diccionario) y un valor de la variable x_0 y evalúe $p(x_0)$

```
def evalpoli(poli,x):                                          ①
    return  sum([poli[m]*x**m for m in poli])                  ②
evalpoli(p,1)                                                  ③
```

① Definimos la función.
② Observe el uso de comprensión de lista. Se recorren las claves del diccionario y se multiplica `valor` por `x` elevado a `clave`.
③ Se hace el llamado a la función `evalpoli`, entregándole el polinomio `p` y el valor donde se avaluará.



Así que el polinomio 1 en $x_0 = 1$, toma un valor de 3, es decir, $p(1) = 3$.

El la parte esencial de la función `evalpoli` es que la variable `m` varía sobre las claves del diccionario, luego se recupera el valor correspondiente a esa clave con `poli[m]`, y se multiplica por `x` elevado a la `m`.

